

Day 13 — SQL Optimization + UPSERT/MERGE

Indexes, Execution Plans, Partitioning & Upserts — Full Detail

Examples continue using customers(customer_id, customer_name, country) and orders(order_id, customer_id, order_date, amount) from Days 10-12.

1. Indexes

An index is a separate data structure that lets the database find rows without scanning the entire table - similar to an index at the back of a book: instead of reading every page, you jump straight to the right one.

Without an index:

```
SELECT * FROM orders WHERE customer_id = 101;
-- Without an index on customer_id, the database checks every single row
-- ("full table scan") to find matches.
```

Creating an index:

```
CREATE INDEX idx_orders_customer_id
ON orders (customer_id);

-- Now the same query can jump directly to matching rows
-- instead of scanning the whole table.
```

The tradeoff - indexes are not free:

- **Faster reads** - SELECT queries filtering on that column speed up significantly
- **Slower writes** - every INSERT/UPDATE/DELETE must also update the index
- **Extra storage** - an index takes disk space on top of the table itself

Key takeaway: *Index columns that are frequently used in WHERE, JOIN, or ORDER BY - not every column. Over-indexing a write-heavy table can hurt performance more than it helps.*

2. Primary/Foreign Keys (Performance Angle)

You already know PK/FK conceptually from data modeling (Day 21). The performance angle: most databases **automatically create an index on the primary key**, but **do not automatically index foreign key columns**.

```
-- customer_id is a foreign key in orders, but has NO automatic index
CREATE TABLE orders (
  order_id INT PRIMARY KEY,
  -- order_id above is automatically indexed (it is the PK)
  customer_id INT REFERENCES customers(customer_id),
  -- customer_id above is NOT automatically indexed
  order_date DATE,
  amount NUMERIC
);

-- You must add this yourself if you join/filter on customer_id often:
CREATE INDEX idx_orders_customer_id ON orders(customer_id);
```

Key takeaway: *If a query joining two tables feels slow, the very first thing to check is whether the foreign key column used in the JOIN condition has an index.*

3. Execution Plans, EXPLAIN & EXPLAIN ANALYZE

Before running a query, the database builds a plan describing exactly how it intends to execute it - which indexes to use, which order to join tables in, whether to scan the whole table or not. You can see this plan yourself.

EXPLAIN - shows the planned strategy without running the query:

```
EXPLAIN
SELECT * FROM orders WHERE customer_id = 101;

-- Output (simplified):
-- Seq Scan on orders (cost=0.00..18.50 rows=5 width=32)
--   Filter: (customer_id = 101)
```

EXPLAIN ANALYZE - actually runs the query and shows real timing:

```
EXPLAIN ANALYZE
SELECT * FROM orders WHERE customer_id = 101;

-- Output includes real numbers:
-- Seq Scan on orders (actual time=0.02..0.45 rows=5 loops=1)
-- Planning Time: 0.10 ms
-- Execution Time: 0.52 ms
```

What to look for in the output:

- **Seq Scan** (sequential scan) - reads the whole table; fine for small tables, a red flag on large ones
- **Index Scan** - used an index to jump to matching rows directly; usually much faster
- **cost** - the planner's own estimate of how expensive a step is (not a real time unit, just a relative number)
- **actual time** - real elapsed time (only appears in EXPLAIN ANALYZE, since it actually executes the query)

Key takeaway: *EXPLAIN shows what the database plans to do; EXPLAIN ANALYZE shows what it actually did, with real timing. Always add an index, then re-run EXPLAIN ANALYZE to confirm the plan actually changed from Seq Scan to Index Scan - never just assume it helped.*

4. Avoiding Unnecessary Columns

Requesting only the columns you actually need - instead of `SELECT *` - reduces the amount of data the database has to read and send back.

```
-- Wasteful - reads and transfers every column, even unused ones
SELECT * FROM orders WHERE customer_id = 101;

-- Efficient - only reads/transfers what is actually needed
SELECT order_id, amount FROM orders WHERE customer_id = 101;
```

Key takeaway: *This matters more as tables get wider (more columns) and larger (more rows) - the cost of pulling unused columns compounds at scale, even though it looks harmless on a small sample table.*

5. Join Optimization

Practical rules for keeping joins fast:

- **Index the join columns** on both sides - as covered in Section 2
- **Filter before joining where possible** - reducing row count with `WHERE` first means less data flows into the `JOIN`
- **Avoid joining on computed/transformed columns** (e.g. `ON LOWER(a.email) = LOWER(b.email)`) - this often prevents the database from using an index at all
- **Watch for the Day 10 duplicate-row problem** - a join exploding row counts isn't just a correctness bug, it's also a performance problem

```
-- Less efficient - filters after joining everything
SELECT c.customer_name, o.amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.order_date >= '2026-01-01';

-- A CTE can make the intent explicit: filter first, then join
WITH recent_orders AS (
    SELECT * FROM orders WHERE order_date >= '2026-01-01'
)
SELECT c.customer_name, ro.amount
FROM customers c
JOIN recent_orders ro ON c.customer_id = ro.customer_id;
```

Key takeaway: *Modern query planners are often smart enough to reorder these automatically, but understanding the logic helps you read an execution plan and know what "should" be happening.*

6. Basic Partitioning

Partitioning physically splits one large table into smaller pieces behind the scenes (commonly by date range), while it still looks like a single table to queries.

```
-- Conceptual example: orders partitioned by year
CREATE TABLE orders (
  order_id INT,
  customer_id INT,
  order_date DATE,
  amount NUMERIC
) PARTITION BY RANGE (order_date);

CREATE TABLE orders_2025 PARTITION OF orders
  FOR VALUES FROM ('2025-01-01') TO ('2026-01-01');

CREATE TABLE orders_2026 PARTITION OF orders
  FOR VALUES FROM ('2026-01-01') TO ('2027-01-01');
```

Why this helps: a query filtering WHERE order_date >= '2026-01-01' only needs to scan the orders_2026 partition - it can skip orders_2025 entirely. This is called **partition pruning**.

Key takeaway: *Partitioning is most valuable on very large, time-series-style tables (logs, transactions, events) where most queries only care about a recent date range - exactly the kind of table you'll build in the warehouse phase (Days 21-27).*

7. UPSERT / ON CONFLICT / MERGE

An upsert means "update if the row already exists, insert if it doesn't" - a single statement instead of writing separate logic to check-then-decide.

PostgreSQL syntax - ON CONFLICT:

```
INSERT INTO customers (customer_id, customer_name, country)
VALUES (101, 'Akash Sharma', 'India')
ON CONFLICT (customer_id)
DO UPDATE SET
  customer_name = EXCLUDED.customer_name,
  country = EXCLUDED.country;

-- If customer_id 101 already exists: updates their name/country
-- If customer_id 101 does not exist: inserts a new row
```

EXCLUDED refers to the row that was *attempted* to be inserted - it lets you reference the new incoming values inside the DO UPDATE clause.

Standard SQL syntax - MERGE (used in many other databases):

```

MERGE INTO customers AS target
USING (
    SELECT 101 AS customer_id,
           'Akash Sharma' AS customer_name,
           'India' AS country
) AS source
ON target.customer_id = source.customer_id
WHEN MATCHED THEN
    UPDATE SET customer_name = source.customer_name, country = source.country
WHEN NOT MATCHED THEN
    INSERT (customer_id, customer_name, country)
    VALUES (source.customer_id, source.customer_name, source.country);

```

Why this matters for data engineering: this is the exact mechanism behind incremental loads (Day 29) and dbt's incremental models (Day 34) - re-running a pipeline should update changed records and insert new ones, not create duplicates.

Key takeaway: *Without an upsert, re-running an INSERT-only pipeline on the same data creates duplicate rows every time. UPSERT/MERGE is what makes a pipeline safely idempotent (Day 29) - re-runnable without corrupting the data.*

Day 13 Summary — SQL Optimization + UPSERT/MERGE

Concept	One-Line Recap
Indexes	Faster reads, slower writes, extra storage - index what you filter/join on
PK/FK Indexing	Primary keys are auto-indexed; foreign keys usually are not
EXPLAIN / ANALYZE	See the planned vs. actual execution strategy and timing
Avoid SELECT *	Only request columns you actually need
Join Optimization	Index join columns, filter early, avoid transformed join conditions
Partitioning	Split large tables physically (e.g. by date) so queries can skip irrelevant chunks
UPSERT / MERGE	Update-if-exists, insert-if-not - the mechanism behind idempotent pipelines

Overall takeaway: *Optimization is not about memorizing tricks - it's about understanding what the database is actually doing (via EXPLAIN) and giving it the right structure (indexes, partitions) to do less work. UPSERT closes the loop from Day 9-12 querying into Day 29's incremental-loading and idempotency concepts.*